Tutorial Sheet: Tree-3

1. Given two max heaps of size n each, what is the minimum possible time complexity to make a one max-heap of size from elements of two max heaps?
   (A) O(nLogn)
   (B) O(nLogLogn)
   (C) O(n)
   (D) O(nLogn)

   Answer: (C)

   Explanation: We can build a heap of 2n elements in O(n) time. Following are the steps.

Create an array of size 2n and copy elements of both heaps to this array.

Call build heap for the array of size 2n. Build heap operation takes O(n) time.

2. Which of the following Binary Min Heap operation has the highest time complexity?
   (A) Inserting an item under the assumption that the heap has capacity to accommodate one more item
   (B) Merging with another heap under the assumption that the heap has capacity to accommodate items of other heap
   (C) Deleting an item from heap
   (D) Decreasing value of a key

   Answer: (B)

   Explanation: The merge operation takes O(n) time, all other operations given in question take O(Logn) time.
   The Binomial and Fibonacci Heaps do merge in better time complexity.

3. Consider any array representation of an n element binary heap where the elements are stored from index 1 to index n of the array. For the element stored at index i of the array (i <= n), the index of the parent is
(A) i – 1
(B) floor(i/2)
(C) ceiling(i/2)
(D) (i+1)/2

Answer: (B)

Explanation: Binary heaps can be represented using arrays: storing elements in an array and using their relative positions within the array to represent child-parent relationships.
For the binary heap element stored at index i of the array,

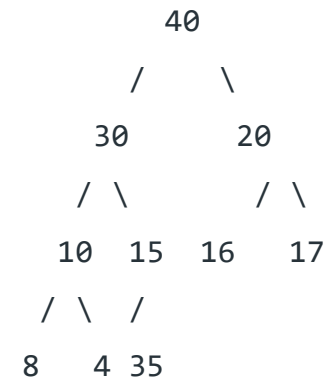
Parent Node will be at index: floor(i/2)
Left Child will be at index: 2i
Right child will be at index: 2*i + 1

4. Consider a max heap, represented by the array: 40, 30, 20, 10, 15, 16, 17, 8, Now consider that a value 35 is inserted into this heap. After insertion, the new heap is
(A) 40, 30, 20, 10, 15, 16, 17, 8, 4, 35
(B) 40, 35, 20, 10, 30, 16, 17, 8, 4, 15
(C) 40, 30, 20, 10, 35, 16, 17, 8, 4, 15
(D) 40, 35, 20, 10, 15, 16, 17, 8, 4, 30

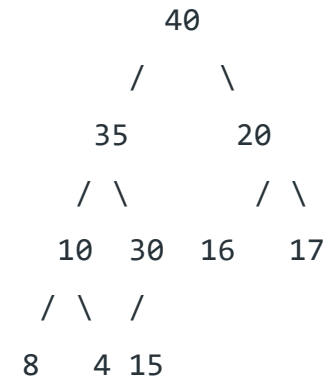
Answer: (B)

Explanation: The array 40, 30, 20, 10, 15, 16, 17, 8, 4 represents following heap

After insertion of 35, we get



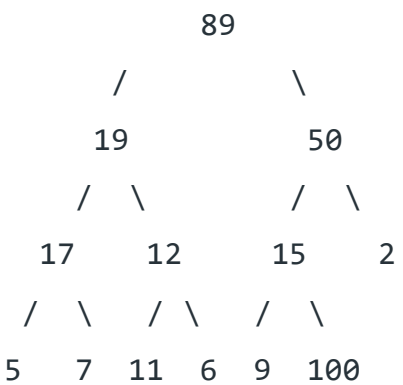
After swapping 35 with 15 and swapping 35 again with 30, we get



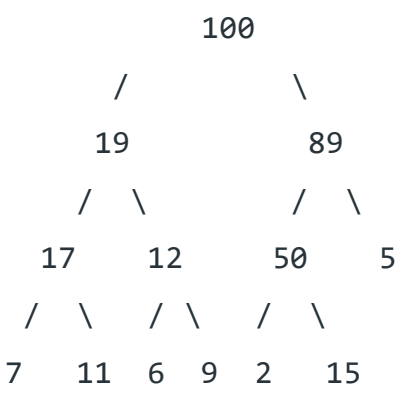
5. Consider the following array of elements. ⟨89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100⟩. The minimum number of interchanges needed to convert it into a max-heap is
   (A) 4
   (B) 5
   (C) 2
   (D) 3

   Answer: (D)

   Explanation: ⟨89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100⟩



```
Minimum number of swaps required to convert above tree
to Max heap is 3. Below are 3 swap operations.
Swap 100 with 15
Swap 100 with 50
Swap 100 with 89
```



6. A complete binary min-heap is made by including each integer in [1, 1023] exactly once. The depth of a node in the heap is the length of the path from the root of the heap to that node. Thus, the root is at depth 0. The maximum depth at which integer 9 can appear is.

(A) 6
(B) 7
(C) 8
(D) 9

Answer: (C)

Explanation: here node with integer 1 has to be at root only. Now for maximum depth of the tree the following arrangement can be taken. Take root as level 1.
make node 2 at level 2 as a child node of node 1.
make node 3 at level 3 as the child node of node 2.
..
.. and so on for nodes 4,5,6,7
..

make node 8 at level 8 as the child node of node 7.
make node 9 at level 9 as the child node of node 8.

Putting other nodes properly, this arrangement of the complete binary tree will follow the property of min heap.

7.

The number of nodes of height h in any n - element heap is _____.

A h

B z^h

C $ceil(\frac{n}{z^h})$

D $ceil(\frac{n}{z^{h+1}})$

Ans: D

8. Consider the array representation of a binary min-heap containing 1023 elements. The minimum number of comparisons required to find the maximum in the heap is _____ .

Note – This question was Numerical Type.
(A) 510
(B) 511
(C) 512
(D) 255

Answer: (B)

Explanation: In a Min-heap having n elements, there are ceil(n/2) leaf nodes.
So, there will be ceil(1023/2) = ceil(511.5) = 512 elements as external nodes.
Now, in general, to find maximum of n elements you need (n-1) comparisons. Just compare first two and then select the larger and compare with next one, select the larger and compare with next one etc.

Therefore, we need 511 (=512 – 1) minimum number of comparisons required to find the maximum in the given heap.

9. Let H be a binary min-heap consisting of n elements implemented as an array. What is the worst case time complexity of an optimal algorithm to find the maximum element in H?
(A) Θ(1)
(B) Θ(logn)
(C) Θ(n)
(D) Θ(nlogn)

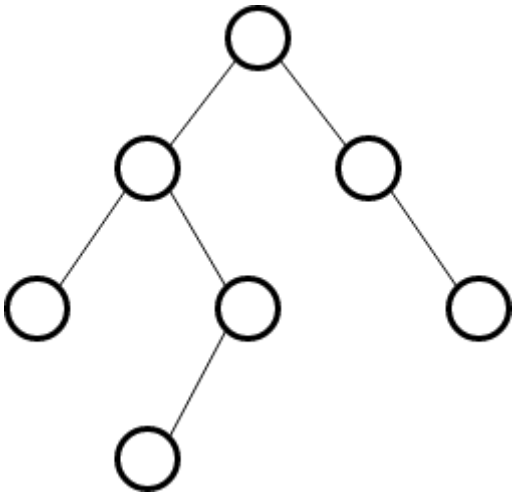
Answer: (C)

Explanation: In case of min heap if we need to find out max element than it should be present at leave nodes so in worst case we need to search till leaf nodes we can't perform binary search here because its not BST and heaps need not be in sorted order so in worst case it would be (n/2)+1. On normalizing it would be O(n) which is option 3.

10. What is the worst case possible height of AVL tree?

- **(A)** 2*logn
- **(B)** 1.44*log n
- **(C)** Depends upon implementation
- **(D)** θ(n)

Solution: The worst case possible height of AVL tree with n nodes is 1.44*logn. This can be verified using AVL tree having 7 nodes and maximum height.



Checking for option (A), 2*log7 = 5.6, however height of tree is 3.

Checking for option (B), 1.44*log7 = 4, which is near to 3.

Checking for option (D), n = 7, however height of tree is 3.

Que.11 Which of the following is TRUE?

- **(A)** The cost of searching an AVL tree is θ(log n) but that of a binary search tree is O(n)
- **(B)** The cost of searching an AVL tree is θ(log n) but that of a complete binary tree is θ(n log n)
- **(C)** The cost of searching a binary search tree is O(log n) but that of an AVL tree is θ(n)
- **(D)** The cost of searching an AVL tree is θ(n log n) but that of a binary search tree is O(n)

Solution: AVL tree's time complexity of searching, insertion and deletion = O(logn). But a binary search tree, may be skewed tree, so in worst case BST searching, insertion and deletion complexity = O(n).

Q.12

In a balanced binary search tree with n elements, what is the worst case time complexity of reporting all elements in range [a,b] ? Assume that the number of reported elements is k.
(A) Θ(log n)
(B) Θ(log(n)+k)
(C) Θ(k log n)
(D) Θ(n log k)

Answer: (B)

Explanation:

1. Time complexity to check if element 'a' in given balanced binary search tree = O(log n)
2. Time complexity to check if element 'a' in given balanced binary search tree = O(log n)
3. Now, time complexity to traverse all element in range [a, b], those elements will be inorder sorted = θ(k)

Therefore, total time complexity will be,

```
= 0(log(n)) + 0(log(n)) + 0(k)
= 0(2(log(n))+k)
= 0(log(n)+k)
```

Q.13

What is the worst case time complexity of inserting n^2 elements into an AVL-tree with n elements initially ?
(A) $\Theta(n^4)$
(B) $\Theta(n^2)$
(C) $\Theta(n^2 \log n)$
(D) $\Theta(n^3)$

Answer: (C)

Explanation: Since AVL tree is balanced tree, the height is O(log n). So, time complexity to insert an element in an AVL tree is O(log n) in worst case.
Note:

```
Every insertion of element:

Finding place to insert = O(log n)

If property not satisfied (after insertion) do rotation = O(log n)


So, an AVL insertion take = O(log n) + O(log n) = O(log n) in worst
case.
```

Now, given n^2 element need to insert into given AVL tree, therefore, total time complexity will be $O(n^2 \log n)$.

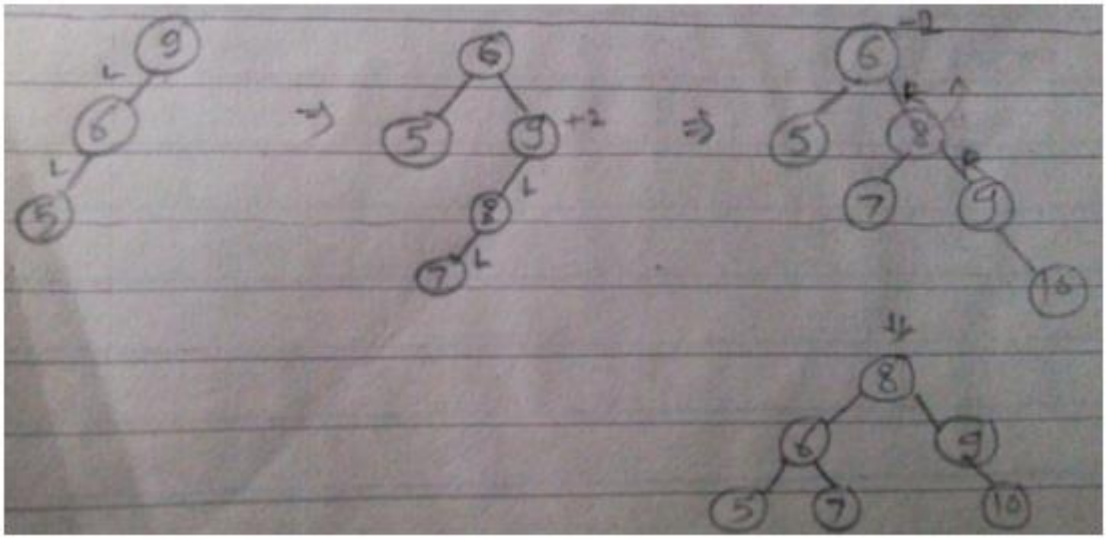
Q.14

The number of rotations required to insert a sequence of elements 9,6,5,8,7,10 into an empty AVL tree is?

A. 0
B. 1
C. 2
D. 3

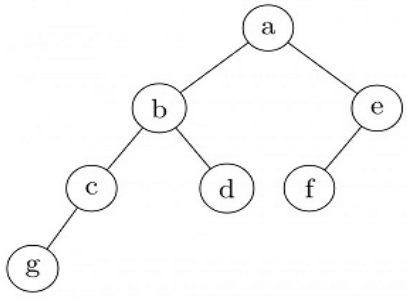
Ans: D

Rotations are : LL LL RR



Q.16

In the balanced binary tree in the below figure, how many nodes will become unbalanced when a node is inserted as a child of the node g?



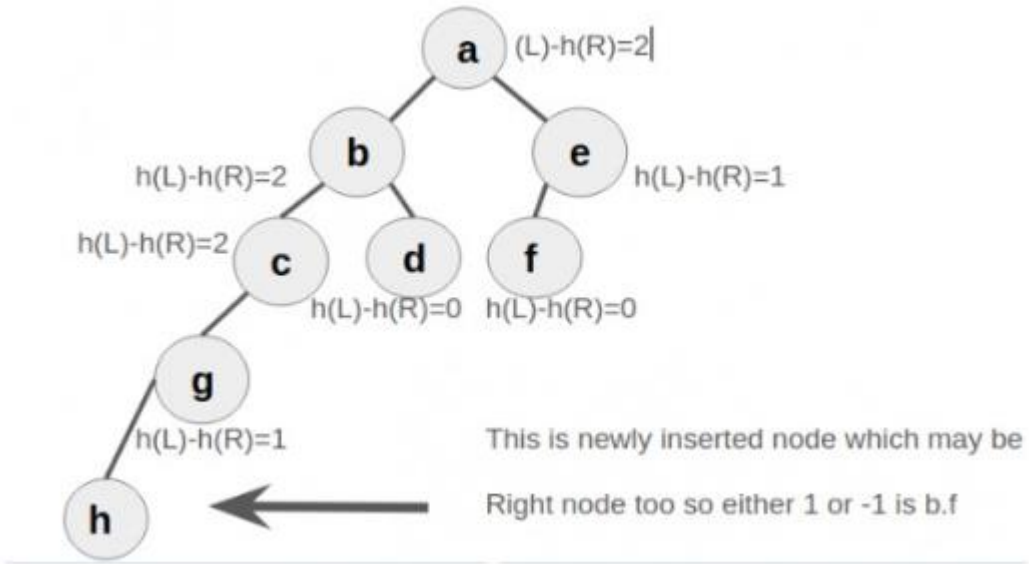
A 1

B 3

C 7

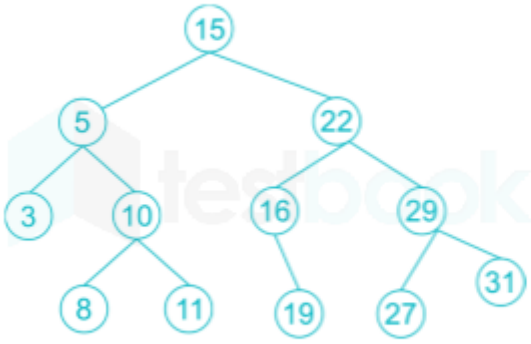
D 8

Ans: B



Q.17

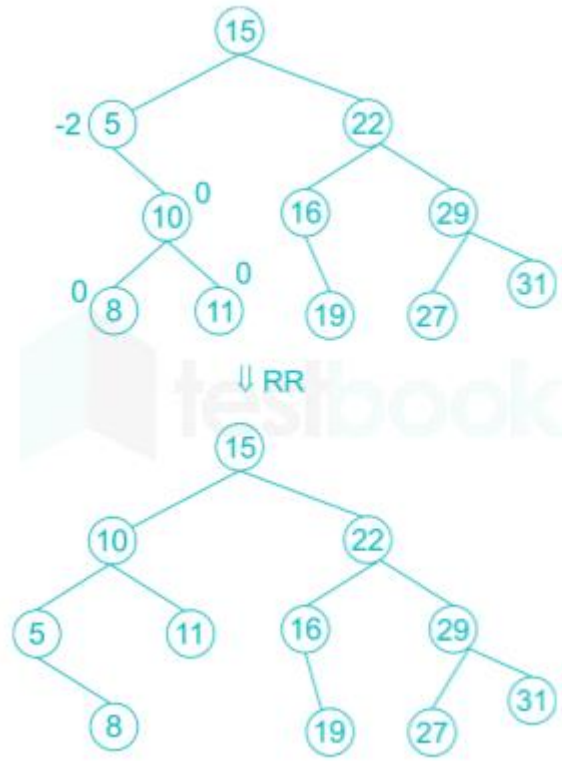
Consider the following AVL tree.



How many number of rotations required for deleting 3?

Solution:

after Deleting 3:



In RR, there is a total of 1 rotation required.